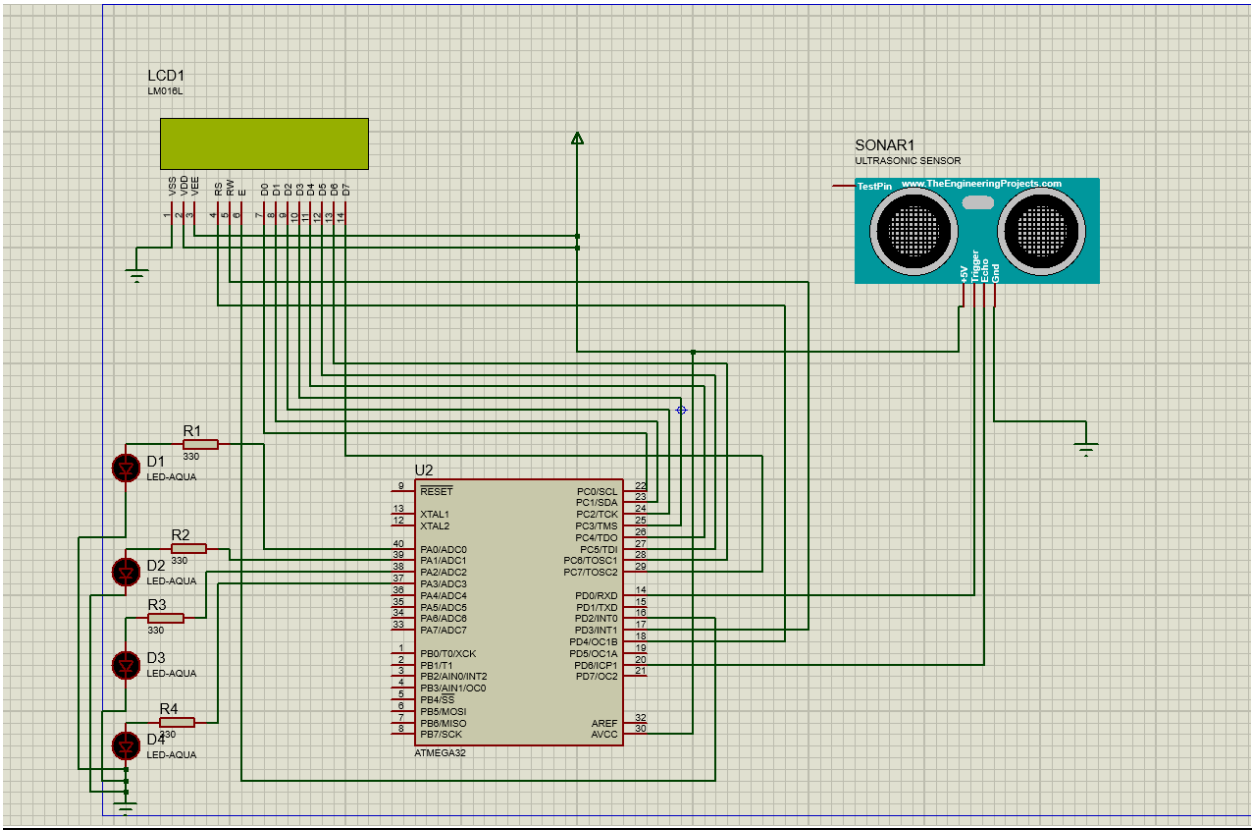
PROJECT

Computer Organization

Laila Tarek 231003520

Circuit Diagram on Porteus:

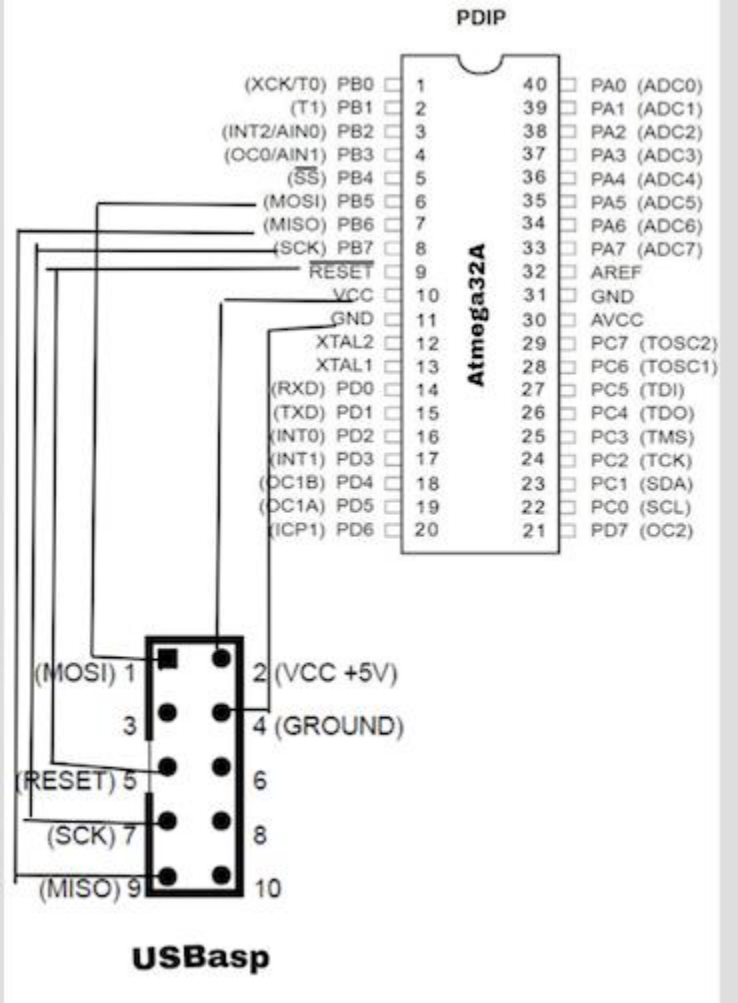


Explanation of connections:

- First ,the connection of the USBasp to the microcontroller :

I didn't connect it on proteus because there is no USBasp but my connection was according to a diagram that I found on Google

I connected the VCC (pin 2)of the USBasp to pin 10 on the Atmega32a and also the ground (pin 4) to pin 11 on the microcontroller , I connected pin 1 (master out slave in) to pin 7 on the atmega32a, the reset (pin 5) connected to pin 9, pin 7 connected to pin 8 on the Atmega and finally pin 9 connected to pin 7 on the Atmega ... the diagram:



- <u>Second, the connection of the LEDs to the microcontroller:</u>

I chose Port A to connect all four LEDs. Specifically, I connected PA0 to the first LED, PA1 to the second LED, PA2 to the third LED, and PA3 to the fourth LED. My goal is as follows:

- When the object is less than 5 cm from the ultrasonic sensor, all 4 LEDs light up.
- When the distance is less than 10 cm, 3 LEDs light up.
- When the distance is less than 15 cm, 2 LEDs light up.
- Finally, when the distance is less than 20 cm, only 1 LED lights up.

- <u>Third, the connection of the Ultrasonic sensor to the microcontroller:</u>

I found on a website that the Echo pin must be connected to ICP1 (PD6), which corresponds to pin 20 on the microcontroller. This is because ICP1 is associated with the Input Capture Unit (ICU), which is essential for accurately measuring the time taken by the ultrasonic signal to travel to and from the object.

To simplify the setup, I chose to connect the Trigger pin to PD0, so that all the pins of the ultrasonic sensor are connected to Port D. Finally, I connected the VCC of the ultrasonic sensor to the VCC of the breadboard and the GND of the ultrasonic sensor to the GND of the breadboard.

- Lastly, the connection of the LCD to the microcontroller:

First, I connected the VSS pin of the LCD to the GND of the breadboard and the VDD pin to the VCC of the breadboard. For the VO pin, I used two 5kΩ potentiometers connected in parallel to achieve better contrast on the LCD for improved visibility.

Next, I decided to connect the RS, RW, and EN control pins to Port D of the microcontroller. This choice was made to keep all 8 bits of Port C free for the LCD's data pins. Specifically, I connected EN to pin PD2, RW to pin PD3, and RS to pin PD4.

Afterward, I connected the LCD's data pins (D0 to D7) directly to the corresponding pins of Port C on the microcontroller (C0 to C7). Finally, I connected the Anode of the LCD's backlight to the VCC of the breadboard and the Cathode to the GND of the breadboard.

The code :

```c
/*
 * LailaaaBS.c
 *
 * Created: 12/15/2024 9:03:22 PM
 * Author : Layla
 */

#define F_CPU 1000000UL   //1Million clock per second

#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>
#include <stdio.h>
#include <string.h>


#define  Trigger_pin    PD0 /* Trigger pin */  // send waves to the object
#define Echo_pin PD6         /* echo pin */   //recieve from the trigger

#define LCD_DATA PORTC   // screen data plogs in
#define en PD2     //
#define rw PD3     // -> to control the screen
#define rs PD4     //

// Global Variables
int TimerOverflow = 0; // Timer overflow counter
uint32_t count = 0; //stores a big number (like how long it took for the sound to come back)
double distance = 0.0;  //3alashan 27seeb el distance
```

114 %

Error List

```c
char string[10];
// ISR for Timer1 Overflow
ISR(TIMER1_OVF_vect) {
    TimerOverflow++;    //lama el register yetmala bel clocks beyzawed el time overflow w ye3ml interrupt fro distance accuracy
}

// Function to Trigger LEDs Sequentially
void light_leds() {
    PORTA |= (1 << PINA0);
    _delay_ms(100);
    PORTA &= ~(1 << PINA0);

    PORTA |= (1 << PINA1);
    _delay_ms(100);
    PORTA &= ~(1 << PINA1);

    PORTA |= (1 << PINA2);
    _delay_ms(100);
    PORTA &= ~(1 << PINA2);

    PORTA |= (1 << PINA3);
    _delay_ms(100);
    PORTA &= ~(1 << PINA3);
}

double ultrasonic2()
{
```

.4 %

```
double ultrasonic2()
{

    // Give 10us trigger pulse on trig. pin to HC-SR04
    PORTD |= (1 << Trigger_pin);   //hena beybda2 el pulse
    _delay_us(10);
    PORTD &= (~(1 << Trigger_pin)); // bey end el pulse 3alasahn yeb3at lel echo

    TCNT1 = 0;  //Clear Timer counter , This is the timer register that counts how much time has passed.
    TCCR1B = 0x41;  // Capture on rising edge, No prescaler, Sets the timer to detect the rising edge of the echo signal.
    TIFR = 1<<ICF1; // Clear ICP flag (Input Capture flag)
    TIFR = 1<<TOV1; // Clear Timer Overflow flag

    //Calculate width of Echo by Input Capture (ICP)

    while ((TIFR & (1 << ICF1)) == 0);  //Wait for rising edge , 3alashan it happens when echo signal starts

    TCNT1 = 0;  //Clear Timer counter
    TCCR1B = 0x01;  // Capture on falling edge, No prescaler
    TIFR = 1<<ICF1; // Clear ICP flag (Input Capture flag)
    TIFR = 1<<TOV1; // Clear Timer Overflow flag
    TimerOverflow = 0; // Clear Timer overflow count , If the timer reaches its maximum value (65535), it resets to 0 w beyzawed fel count

    while ((TIFR & (1 << ICF1)) == 0); // Wait for falling edge

    count = ICR1 + (65535 * TimerOverflow); // Take count
    // 8MHz Timer freq, sound speed =343 m/s
    distance = (double)count / 58.0;

    return distance;

}
```

4 %

```c
void LCD_cmd(unsigned char cmd)
{
    LCD_DATA = cmd;
    PORTD &= ~(1<<rs); //register select(cmd aw data) , 0 beteb3at cmd w law 1  beteb3at data,
    PORTD &= ~(1<<rw); // law 0 the microcontroller is writing data to the lcd , law 1 bey read
    PORTD |=(1<<en); // lama en yefdal ye3ml high then low bey read el data el mab3otalo
    _delay_ms(5);
    PORTD &= ~(1<<en);

}

void LCD_write(unsigned char data)
{
    LCD_DATA = data;
    PORTD |= (1<<rs); //1 keda beyb3at data
    PORTD &= ~(1<<rw); //0 beysend cmd
    PORTD |= (1<<en);
    _delay_ms(5);
    PORTD &= ~(1<<en);
}


void init_LCD(void)
{
    LCD_cmd(0x38); //This command is necessary to initialize the LCD in the correct operating mode for a typical 16x2 display
    _delay_ms(5);
    LCD_cmd(0x01); //clear dislpay
    _delay_ms(5);
    LCD_cmd(0x02); //   This command moves the cursor to the home position
    _delay_ms(5);
    LCD_cmd(0x06); //is the entry mode set command
    _delay_ms(5);
    LCD_cmd(0x80); //This command sets the cursor position at the first position on the first line of the display (row 1, column 1)
    _delay_ms(5);
}
```

94 %

```c
void display(double distance){

    LCD_cmd(0x01); //hay clear

    char laila[16]= "*Distance=";
    char dist_string[5];
    sprintf(dist_string, "%.0f", distance); // Convert the float to string
    char ba2it_el_ketaba[3]  = "cm*";
    for(int i =0; i<10; i++)
    {
        LCD_write(laila[i]);
        _delay_ms(1);
    }
    int size_of_distt = sizeof(dist_string)/sizeof(dist_string[0]); // calculates the number of characters in dist_string
    for(int i =0; i<size_of_distt;i++)
    {
        LCD_write(dist_string[i]);
        _delay_ms(1);
    }

    for(int i=0; i<3;i++)
    {
        LCD_write(ba2it_el_ketaba[i]);
        _delay_ms(1);
    }

    LCD_cmd(0xC0); //3alashan anzel 3ala new line
    _delay_ms(1);
    LCD_cmd(0x0C); // keda ba3ml display on , cursor off
    char BS[15] = "*By Laila BS*<3";
    for(int i = 0 ; i<15; i++)
    {
        LCD_write(BS[i]);
        _delay_ms(1);
    }


}

int main(void) {
```

```c
int main(void) {
    // Configure Ports

    DDRA = 0xFF;  // PORTA as Output (LEDs)
    PORTA = 0x00; // Initialize PORTA LEDs off

    DDRD |= (1 << PIND0); // PC0 as Output (Trigger Pin)
    DDRD &= ~(1 << PIND6); // PD6 as Input (Echo Pin)

    DDRD |= (1 << PIND2); // these are for the commands en , rs, rw
    DDRD |= (1 << PIND3);
    DDRD |= (1 << PIND4);
    DDRC = 0xFF;  //this is for the data

    init_LCD();
    _delay_ms(5);

    LCD_cmd(0x0C);
    _delay_ms(5); // 3alashan  just in case el lcd me3afena

    // Initialize Timer1
    sei();                  // Enable Global Interrupts
    TIMSK = (1 << TOIE1);   // Enable Timer1 Overflow Interrupt
    TCCR1A = 0;             // Normal Mode

    double dist;

    while (1) {
        //light_leds();


        dist = ultrasonic2(); // Measure Distance

        // LED Control Logic Based on Distance
        if (dist < 5) {
            PORTA |= (1 << PINA0) | (1 << PINA1) | (1 << PINA2) | (1 << PINA3);


        }
        else if (dist < 10) {
            PORTA |= (1 << PINA0) | (1 << PINA1) | (1 << PINA2);
```

'8 %

```
    while (1) {
        //light_leds();


        dist = ultrasonic2(); // Measure Distance

        // LED Control Logic Based on Distance
        if (dist < 5) {
            PORTA |= (1 << PINA0) | (1 << PINA1) | (1 << PINA2) | (1 << PINA3);


        }
        else if (dist < 10) {
            PORTA |= (1 << PINA0) | (1 << PINA1) | (1 << PINA2);
            PORTA &= ~(1 << PINA3);

        }
        else if (dist < 15) {
            PORTA |= (1 << PINA0) | (1 << PINA1);
            PORTA &= ~((1 << PINA2) | (1 << PINA3));
        }
        else if (dist < 20) {
            PORTA |= (1 << PINA0);
            PORTA &= ~((1 << PINA1) | (1 << PINA2) | (1 << PINA3));
        }
        else {
            PORTA &= ~((1 << PINA0) | (1 << PINA1) | (1 << PINA2) | (1 << PINA3));

        }
        display(dist);

        _delay_ms(50);  // Small Delay Between Measurements
        //PORTA &= ~((1 << PINA0) | (1 << PINA1) | (1 << PINA2) | (1 << PINA3));
        //_delay_ms(100);
    }
}
```